What-If Witness Spaces: A Neuro-Symbolic Disaster Loop for Fail-Closed Software Hardening

Dana Edwards

Independent Researcher

ORCID: https://orcid.org/0009-0001-1177-4752

Preprint, version 1.0

Date: 2026-04-25

DOI: Zenodo DOI pending first release

Repository DOI badge URL:

https://zenodo.org/badge/latestdoi/1113028896

Technique: What-if witness spaces and neuro-symbolic disaster loops

Case-study artifacts: MPRD and ZenoDEX

License: CC BY 4.0

Reference implementation: `tools/run_neuro_symbolic_disaster_loop.py`
Case-study receipt: `docs/whitepapers/neuro_symbolic_case_study/neuro_symbolic_disaster_loop_latest.json`
Latest case-study receipt hash at time of writing: `sha256: b83ee1178c4bea460b9c7e3c67d5ccf1d7ac330e017746236fcfc7274c459561`
Recommended citation before Zenodo DOI minting: Edwards, Dana. 2026. "What-If Witness Spaces: A Neuro-Symbolic Disaster Loop for Fail-Closed Software Hardening." Preprint, version 1.0. Zenodo DOI pending.

Contents

Abstract

Modern assurance fails most often at composition boundaries. A parser, proof, fuzz target, or model checker may be correct in isolation while the larger workflow still reaches a disaster state through stale evidence, retry paths, optional features, provenance drift, proof-journal mismatch, or promotion from an unsafe research state. This paper introduces **what-if witness spaces**: a general neuro-symbolic technique in which a creative proposer generates explicit disaster hypotheses and a deterministic symbolic checker decides whether each hypothesis is blocked, refuted, or reachable under replayable evidence.

The method generalizes counterexample-guided reasoning from local program inputs to the assurance workflow itself. The neural side is an existential generator: it asks "what if?" over surfaces, graph edges, evidence states, and fault mutations. The symbolic side is a universal gate: it checks every generated witness in a bounded language, rejects unknown or stale evidence fail-closed, and opens research only when all obligations are discharged. The mathematical core comes from witness-space factorization, quantifier factoring, Galois-style obligation carving, verifier-compiler loops, and loop-space geometry.

The first case study is MPRD, where the loop checked 16,003 materialized what-ifs plus a compressed 75,599,999-combination independent frontier, found 0 reachable disaster witnesses, observed 0 fail-closed failures across receipt, blocker, provenance, and optional-conflict mutations, and opened the bounded research gate. The second case study is ZenoDEX, a financial protocol and DEX assurance surface where the same loop shape is applied to loss-bearing witness spaces: settlement certificates, oracle freshness, nonce replay, integer rounding, sealed-bid terminal states, confidential receipts, proof gates, and claim-evidence routing. The claim is intentionally bounded: it proves no global software safety theorem. It proves a replayable no-witness result for a named witness language, atlas, receipt set, checker, and depth, and it shows how the same discipline generalizes to financial mechanisms.

1 The Assurance Gap

Most software hardening workflows are organized around local artifacts:

- unit tests for local invariants,
- fuzz targets for parsers and state machines,
- concolic or symbolic checks for branchy code,
- proof files for selected mathematical claims,
- CI gates for regressions.

These are necessary, but they do not fully model the composed assurance process. Real failures often arise between artifacts:

- a receipt is valid for an old checker but reused after the checker changes,
- a proof journal is replayed against a different decision token,
- a retry path re-enters replay clearance after side effects,
- a stable optional-lane receipt disagrees with the atlas,
- a promotion gate treats "no bug found" as "safe to research."

These are not only code bugs. They are **disaster states** in the process that decides which states may be explored, optimized, or promoted. The key question is therefore:

Which bad composed states are reachable under the current evidence state?

The answer should not depend on whether a language model is confident. It should depend on a finite, reproducible witness space and a deterministic checker.

2 The Technique

A **what-if witness space** is a finite or compressed symbolic family of hypotheses. A hypothesis names:

- the surfaces involved,
- the disaster state being tested,
- the composition, ordering, re-entry, or evidence-fault pattern,
- the receipt or checker evidence required to decide it.

The loop has two roles:

1. The proposer generates hypotheses. This can be an LLM, a human reviewer, a grammar, a fuzzer, or another search policy. It is creative, but not trusted.
2. The checker classifies hypotheses. It is deterministic, receipt-backed, and fail-closed. It owns promotion authority.

The operational rule is:

Creativity belongs on the existential side; trust belongs on the universal side.

The proposer searches for possible witnesses. The checker decides which witnesses are valid, blocked, or refuted. If the checker returns `UNKNOWN`, `TIMEOUT`, `INCONCLUSIVE`, malformed input, stale evidence, or missing evidence, the gate rejects.

3 Mathematical Foundations

This section distills the mathematical machinery developed in the Formal Methods Philosophy tutorial sequence on neuro-symbolic witness spaces, quantifier factoring, Galois loops, verifier-compiler loops, loop-space geometry, counterexample-guided requirements discovery, grammar-based fuzzing, and concolic branch exploration.

3.1 Witness Spaces

Let x be an assurance state. It contains code, specifications, receipts, provenance, optional guidance, checker state, and a bounded search depth. Let L be a witness language and z a concrete witness

object generated by that language. A local witnessability target is:

$$\mathrm{Good}(x) \iff \exists z\ \mathrm{Proves}_L(z, x).$$

For hardening, the more useful dual is a no-bad-witness target:

$$\mathrm{NoBadWitness}(x, L) \iff \neg\exists z \in L(x)\ \mathrm{Bad}(z, x).$$

The technique does not search the full semantic universe. It searches $L(x)$, the bounded witness language currently declared by the artifact. A no-witness result is therefore:

$$\neg\exists z \in L_d(x)\ \mathrm{ReachableDisaster}(z, x),$$

where d is the configured depth. This is meaningful because L_d is explicit and replayable, not because it is universal.

3.2 Neuro-Symbolic Filtering

Let $q_N(z \mid x)$ be the proposer's distribution over candidate witnesses, and let $\chi_S(z, x)$ be the symbolic admissibility predicate:

$$\chi_S(z, x) = \begin{cases} 1 & \text{if the checker accepts } z \text{ for } x, \\ 0 & \text{otherwise.} \end{cases}$$

The neuro-symbolic distribution is:

$$q_{NS}(z \mid x) \propto q_N(z \mid x) \cdot \chi_S(z, x).$$

This equation captures the division of labor. The proposer concentrates search mass. The checker zeros out semantically invalid mass. The combined loop searches the admissible overlap. If the admissible mass is zero, the distribution is not renormalized into a pass; the gate reports that no admissible candidate was found.

3.3 Quantifier Factoring

Many assurance goals have the form:

$$\exists a\ \forall e\ \mathrm{Spec}(a, e),$$

where a is a design, repair, policy, invariant, or research state, and e is an environment move, attack, execution, input, or evidence perturbation. The central factoring move is:

$$\forall e\ \mathrm{Spec}(a, e) \iff \neg\exists e\ \neg\mathrm{Spec}(a, e).$$

Thus acceptance is operationalized as failed counterexample search:

$$\mathrm{Good}(a) := \forall e\ \mathrm{Spec}(a, e).$$

$$\text{Accept}(a) \iff \neg\exists e\ \text{EmitCE}(a, e).$$

Counterexample soundness is:

$$\forall a, e\ (\text{EmitCE}(a, e) \to \neg\text{Spec}(a, e)).$$

Counterexample completeness is:

$$\forall a\ (\neg\text{Good}(a) \to \exists e\ \text{EmitCE}(a, e)).$$

Only with both soundness and completeness does acceptance equal goodness. In bounded engineering artifacts, completeness is usually limited to the declared frontier. Therefore the honest claim is bounded acceptance, not global safety.

3.4 Galois-Style Obligation Carving

The loop can be expressed as a Galois connection between candidate states and obligations. Let X be candidate states and Y be obligations, and let $\text{Spec} : X \times Y \to \{\text{true}, \text{false}\}$ be the Boolean incidence relation. For $B \subseteq Y$ and $C \subseteq X$, define:

$$\Phi(B) = \{x \in X \mid \forall b \in B,\ \text{Spec}(x, b)\},$$
$$\Psi(C) = \{y \in Y \mid \forall x \in C,\ \text{Spec}(x, y)\}.$$

Then:

$$C \subseteq \Phi(B) \quad \iff \quad B \subseteq \Psi(C).$$

This means candidate-space reasoning and obligation-space reasoning are dual. The closure operators are:

$$\text{cl}_X(C) = \Phi(\Psi(C)), \qquad \text{cl}_Y(B) = \Psi(\Phi(B)).$$

A fail-closed assurance loop maintains surviving candidates C_t, uncovered obligations U_t, and discharged obligations $D_t = Y \setminus U_t$ with the invariant:

$$D_t \subseteq \Psi(C_t).$$

Every discharged obligation must be satisfied by every surviving candidate. A new bad witness shrinks C_t. A new region certificate shrinks U_t. A missing receipt or inconclusive verifier result does not shrink the obligation set; it keeps the gate closed.

3.5 Obligation-Targeted Witness Routing

A stronger version of the loop is not "the proposer guesses an answer and the checker attacks it." This architecture lets the symbolic side choose the highest-value unresolved obligation, then asks the existential side to route the search toward a witness that exposes it.

For an uncovered obligation y, define:

$$W(C, y) = \{x \in C \mid \neg\mathrm{Spec}(x, y)\}.$$

If $y \notin \Psi(C)$, then $W(C, y)$ is nonempty. A closure-gain score is:

$$\Delta_\Psi(y \mid C) = |\Psi(C \cap \Phi(\{y\}))| - |\Psi(C)|\,.$$

A live-burden score is:

$$L(C) = |C| + |Y \setminus \Psi(C)|.$$

On finite bounded domains, an obligation-targeted controller can choose y by maximizing closure gain or minimizing live burden, then require the proposer to synthesize an exposing witness in $W(C, y)$. This is a deeper neuro-symbolic loop: the formal side chooses the leverage-bearing question, and the creative side supplies a concrete route to it.

3.6 Verifier-Compiler View

Let $L^\star : X \to \Lambda$ be an exact verifier label function over a bounded domain. A verifier-compiler loop searches for a quotient:

$$q : X \to Q$$

such that:

$$\forall x, x' \in X,\ q(x) = q(x') \to L^\star(x) = L^\star(x').$$

If this holds, there exists a compiled controller on the image of q:

$$C : q(X) \to \Lambda$$

with:

$$\forall x \in X,\ C(q(x)) = L^\star(x).$$

If q is too coarse, the loop adds a repair coordinate $r : X \to R$ and checks:

$$(q(x), r(x)) = (q(x'), r(x')) \to L^\star(x) = L^\star(x').$$

The disaster loop uses this idea at the assurance level. It does not need every byte of the repository. It needs a quotient that preserves the gate-relevant observations: surfaces, edges, receipts, optional status, blocker references, provenance, checker cleanliness, and depth.

3.7 Loop-Space Geometry

The loop-space geometry view treats the assurance workflow as a state machine. A state contains code, receipts, checker logic, provenance, optional guidance, and gate decisions. A transition is an edit, rerun, receipt mutation, synthetic fault, or witness-space expansion.

For a witness library W and hidden target M, define the observation map:

$$O_W(M) = \{S \in W \mid S \subseteq M\}.$$

Two targets are indistinguishable to the current loop when:

$$M \sim_W M' \quad \Longleftrightarrow \quad O_W(M) = O_W(M').$$

A stronger loop changes this quotient. It collects witnesses, stores the right state, collapses ambiguity classes, asks only the residual questions that still matter, and compiles the surviving structure into a small controller or gate.

For hardening, a disaster loop is a cycle:

$$x_0 \to x_1 \to \cdots \to x_k \quad \wedge \quad x_k \sim_{\text{visible}} x_0 \quad \wedge \quad \text{ObligationLost}(x_0, x_k).$$

The method blocks such loops by making obligations visible. A stale receipt, dirty checker, failed optional lane, or missing blocker changes the quotient and therefore cannot silently return to the same researchable state.

3.8 Loss-Bearing Witness Spaces

Financial protocols add a useful refinement. A disaster is not only an invariant violation; it can be a reachable state with positive extractable value, stalled settlement, stale oracle use, replay acceptance, rounding leakage, or unbound receipt execution. Let V be a value accounting map and let AllowedFlow be the value flow explicitly authorized by the protocol rules. Define the unexplained value movement of a witness by:

$$\text{Leak}(z, x) = \max\{0,\ V_{\text{after}}(z, x) - V_{\text{before}}(x) - \text{AllowedFlow}(z, x)\}.$$

A financial disaster witness is:

$$\begin{aligned} \text{LossWitness}(z, x) \Longleftrightarrow\ & \text{Reachable}(z, x) \\ & \wedge\ (\text{BreaksConservation}(z, x) \vee \text{AcceptsReplay}(z, x) \\ & \quad \vee\ \text{ExecutesStaleOracle}(z, x) \vee \text{ReceiptUnbound}(z, x) \\ & \quad \vee\ \text{RoundingLeak}(z, x) \vee \text{StuckValueState}(z, x)) \\ & \wedge\ \text{Leak}(z, x) > 0. \end{aligned}$$

The standard reading is: a witness matters financially when it is reachable, hits a bad mechanism class, and moves value outside the allowed accounting rules. This separates "the proof has not covered this case yet" from "this case can lose money." Both can block promotion, but they carry different remediation work.

For a financial surface, the witness language decomposes into typed fibers:

$$L_{\text{fin}} = L_{\text{exec}} \cup L_{\text{acct}} \cup L_{\text{oracle}} \cup L_{\text{receipt}} \cup L_{\text{privacy}} \cup L_{\text{liveness}} \cup L_{\text{governance}}.$$

Each fiber has its own checker shape. Arithmetic conservation may route to a proof assistant or SMT solver. Oracle freshness may route to a temporal model and runtime gate. Sealed-bid deadlocks may route to state-machine replay. Receipt binding may route to deterministic packet verification. The loop is stronger when the witness language records this routing explicitly instead of treating all "what ifs" as one undifferentiated fuzz queue.

3.9 Claim-Evidence Routing

The ZenoDEX case study also motivates a claim-evidence graph. Let C be promoted claims, E_v be evidence artifacts, and $R \subseteq C \times E_v$ connect claims to replayable checks. A claim is promotable only when every required evidence class has a passing artifact:

$$\begin{aligned} \text{Promotable}(c) \iff & \text{Status}(c) \in \{\text{supported}, \text{proved}\} \\ & \wedge\ \forall k \in K(c)\ \exists e \in E_v\ .\ R(c, e) \\ & \wedge\ \text{Kind}(e) = k \wedge \text{Pass}(e). \end{aligned}$$

If any required evidence is missing, stale, disputed, inconclusive, or tied to private scratch state that cannot be replayed, the public claim does not promote. This is the same neuro-symbolic loop at a higher layer: the proposer suggests a claim and its expected evidence route; the checker treats the claim-evidence graph as state and rejects unsupported promotion.

4 Generic Algorithm

The method can be implemented with the following abstract interface.

```
input:
  surfaces S
  composition edges E
  re-entry edges Q
  disaster predicates D
  mandatory evidence Rm
  optional evidence Ro
  blocker references B
  provenance state P
  depth d

generate:
  W_mat:
    single-surface disasters
    evidence fail-open mutations
    blocker bypasses
    edge compositions
    order inversions
    bounded chains
    terminal chain disasters
```

```
    fan-out and convergence cases
    re-entry and cycle amplification cases
    independent co-reachability cases

  W_cmp:
    compressed independent frontier above the materialized order

mutate:
  receipt-state faults
  blocker-state faults
  provenance faults
  optional source-of-truth conflicts

classify:
  for every w in W_mat:
    REACHABLE_DISASTER_WITNESS
    UNKNOWN_BLOCKED
    NO_REACHABLE_WITNESS_BOUNDED

gate:
  open only if:
    no reachable disaster witnesses
    no unknown mandatory blockers
    no optional research blockers
    graph frontier exhausted
    synthetic fault checks reject fail-closed
    receipt provenance has no hard mismatch
    gate-critical checker state is clean
```

The generic gate can be written:

$$\begin{aligned}\mathrm{Open}(A,\varepsilon,d) \iff \forall w \in W_{\mathrm{mat}}(A,d)\ .\ \rho(w,\varepsilon)\\ \wedge\ \mathrm{CompressedFrontierOK}(W_{\mathrm{cmp}},\varepsilon)\\ \wedge\ \mathrm{DepthExhausted}(A,d)\\ \wedge\ \forall m \in M\ .\ \mathrm{Reject}(m(\varepsilon)).\end{aligned}$$

Here A is the atlas, ε is the evidence state, ρ is the researchability predicate, and M is the synthetic mutation suite.

5 Case Study: MPRD

MPRD is used here as a case study, not as the definition of the technique. In the case study, the atlas is:

$$A = (S, E, Q, D),$$

where S is the finite set of surfaces, $E \subseteq S \times S$ is the composition graph, $Q \subseteq S \times S$ is the re-entry graph, and $D(s)$ is the finite set of named disaster states attached to surface s.

The evidence state is:

$$\varepsilon = (R_m, R_o, B, H, g, \Delta),$$

where R_m is the mandatory receipt set, R_o is the optional receipt set, B is the blocker-source relation, H is the harness-reference relation, g is the current git head, and Δ is checker worktree/provenance status.

The current case-study atlas covers 11 surfaces:

Surface	Example disaster states
`replay_nonce_claim`	duplicate side effects, nonce claimed twice, distributed claim race
`quorum_snapshot_attestation`	untrusted threshold, duplicate signer, drifted snapshot
`orchestrator_pipeline_ordering`	verify-fail-but-execute, record-before-verify, duplicate payload
`selector_candidate_family`	noncanonical family, out-of-bounds selection, hash mismatch
`operator_control_lifecycle`	invalid retention mutation, mode transition drift
`decision_token_proof_journal_binding`	decision commitment drift, journal state drift
`policy_artifact_run_lifecycle`	unauthorized artifact, source mapping gap, tamper
`registry_key_rotation_authorization`	insufficient quorum, rotated key still authorizes
`tau_policy_certification_boundary`	unsupported policy fail-open, tampered Tau certifies
`executor_side_effect_boundary`	invalid boundary reaches side effect, idempotency drift
`executor_transport_boundary`	invalid boundary reaches network, retry budget drift

The materialized case-study witness language is:

$$\begin{aligned} W_{\mathrm{mat}}(A,d) = & W_{\mathrm{single}} \cup W_{\mathrm{receipt}} \cup W_{\mathrm{blocker}} \cup W_{\mathrm{edge}} \cup W_{\mathrm{order}} \\ & \cup W_{\mathrm{chain}} \cup W_{\mathrm{fan}} \cup W_{\mathrm{conv}} \cup W_{\mathrm{reentry}} \cup W_{\mathrm{cycle}} \cup W^{\leq 3}_{\mathrm{independent}}. \end{aligned}$$

The compressed frontier is:

$$W_{\mathrm{cmp}}(A,d) = \{I \subseteq S \mid 4 \leq |I| \leq d,\ I \text{ is independent under the atlas relations}\}.$$

6 Case-Study Results

At the latest recorded MPRD case-study run:

- gate: `OPEN_FOR_BOUNDED_RESEARCH`
- materialized hypotheses: 16,003
- reachable disaster witnesses: 0
- unknown mandatory blockers: 0
- compressed independent frontier: 75,599,999 combinations
- optional research blocks: 0
- receipt-state synthetic checks: 66, failures 0
- blocker-state synthetic checks: 22, failures 0
- provenance synthetic checks: 16, failures 0
- optional-conflict synthetic checks: 25, failures 0
- hard receipt git mismatches: 0
- compatible checker-only receipt drifts: 12
- checker worktree dirty: false
- compact/full stable hash parity: passed

The materialized family counts were:

Family	Count
`single_surface_disaster`	47
`receipt_fail_open_mutation`	11
`blocker_bypass_disaster`	194
`edge_composition_disaster`	196
`order_inversion_disaster`	196
`chain_researchability`	71
`chain_terminal_disaster`	1,079
`fanout_composition_disaster`	64
`convergence_composition_disaster`	299
`reentry_retry_disaster`	66
`cycle_amplification_disaster`	66
`independent_pair_coreachability`	1,000
`independent_triple_coreachability`	12,714

The bounded case-study theorem is:

$$\mathrm{Open}(A, \varepsilon, 11) \Rightarrow \neg\exists w \in W_{\mathrm{mat}}(A, 11)\ \mathrm{ReachableDisaster}(w, \varepsilon).$$

It is not:

$$\neg\exists w \in W_\infty(A)\ \mathrm{ReachableDisaster}(w, \varepsilon).$$

7 Case Study: ZenoDEX Financial-Risk Hardening

ZenoDEX is used here as an additional case study because financial protocols make the stakes of compositional disaster states concrete. The value-moving surface includes batch settlement, quote and settlement receipts, oracle freshness, nonce sequencing, rounding and dust handling, proof-gated settlement updates, perpetual-market epoch settlement, confidential extension receipts, and sealed-bid auction terminal states.

The useful abstraction is:

$$x_{\mathrm{dex}} = (\sigma_{\mathrm{ledger}}, \sigma_{\mathrm{oracle}}, \sigma_{\mathrm{receipt}}, \sigma_{\mathrm{claim}}, \sigma_{\mathrm{proof}}, \sigma_{\mathrm{time}}).$$

The standard reading is: the DEX assurance state is not just balances and pools. It also includes oracle time, receipt roots, promoted claims, proof/binding state, and temporal queue state. A what-if loop must perturb all of these coordinates, because many DeFi failures occur in the composition of otherwise reasonable modules.

The ZenoDEX evidence structure inspected for this paper includes:

- a tool-agnostic claims registry that marks promoted, supported, proved, and disputed claims with replay commands,
- public replay lanes that separate tracked manifests and checker scripts from private scratch evidence,
- proof-carrying settlement witness kernels for exact-in and exact-out settlement,
- deterministic tests for minimal economic witnesses such as rounding leakage and reward-subsidized oracle manipulation,
- machine-checked proofs for nonce replay, canonical route selection, rounding, CPMM settlement, and batch-auction canonicalization,
- bounded temporal models for oracle recovery, settlement witness lifecycle, exact-out fallback liveness, liquidation queue drain, and settlement witness inclusion under bounded ingress, reorg, fee-priority, and builder-pressure assumptions,
- a sealed-bid disaster-state catalog covering empty-auction, no-reveal, and empty-bond terminal paths,
- confidential extension and FHE sealed-bid alpha boundaries that require freshness, binding, replay protection, local output bounds, and explicit fallback paths.

This adds three lessons beyond the MPRD case study.

First, financial witness languages must include value functions. A state can be syntactically valid and still be unacceptable if it enables positive extraction or creates value outside authorized accounting:

$$\exists z \in L_{\text{fin}}(x) \,.\, \text{Reachable}(z, x) \wedge \text{Leak}(z, x) > 0 \quad \Rightarrow \quad \text{RejectPromotion}(x).$$

Second, liveness can be a safety property when value is locked or risk keeps accumulating. A "safe" fail-closed state that never settles can become a financial disaster. Therefore the loop must ask not only "can a bad transition execute?" but also "can an accepted witness remain unresolved past the bounded fairness assumptions?"

Third, public claims need explicit exclusion. ZenoDEX's assurance surface distinguishes release-backed, public-replay, disputed, experimental, and out-of-scope surfaces. This is essential for publishable assurance: an incomplete proof lane should become an honest boundary, not a hidden global claim.

The bounded ZenoDEX-style gate can be written:

$$\begin{aligned} \text{Open}_{\text{fin}}(x, d) \iff & \neg \exists z \in L_{\text{fin},d}(x) \,.\, \text{LossWitness}(z, x) \\ & \wedge \ \forall c \in C_{\text{promoted}} \,.\, \text{Promotable}(c) \\ & \wedge \ \forall c \in C_{\text{disputed}} \,.\, \neg \text{ReleaseAuthoritative}(c) \\ & \wedge \ \text{ReplayLanesGreen}(x) \wedge \text{PrivateScratchExcluded}(x). \end{aligned}$$

This is a financial version of the same technique. The loop does not claim that a live DEX is universally safe. It claims that, for a named bounded surface, no generated loss-bearing witness survives replay; every promoted claim has its required evidence route; disputed claims stay excluded; and private or experimental tooling is not part of the public proof.

8 Replication Protocol

The public replication path has three tiers for the archived MPRD case study, plus an optional ZenoDEX case-study replay when the ZenoDEX repository or archive is available.

8.1 Tier 1: Paper and Checker Availability

Clone the repository archive associated with the Zenodo release:

```
git clone https://github.com/TheDarkLightX/MPRD.git
cd MPRD
```

Confirm that the checker exists:

```
python3 tools/run_neuro_symbolic_disaster_loop.py --help
```

Run the in-process fail-closed self-tests:

```
python3 tools/run_neuro_symbolic_disaster_loop.py --self-test
```

This tier checks the public checker logic without requiring the full MPRD case-study evidence bundle.

8.2 Tier 2: Case-Study Receipt Replay

The case-study receipt is `neuro_symbolic_disaster_loop_latest.json` in `docs/whitepapers/neuro_symbolic_case_study/`.

When the Zenodo release includes the MPRD assurance bundle, verify the recorded stable hash:

```
python3 - <<'PY'
import json
from pathlib import Path
p = Path("docs/whitepapers/neuro_symbolic_case_study")
p = p / "neuro_symbolic_disaster_loop_latest.json"
d = json.loads(p.read_text())
print(d["stable_receipt_hash"])
print(json.dumps(d["summary"], indent=2, sort_keys=True))
print(json.dumps(d["gate_summary"], indent=2, sort_keys=True))
PY
```

The expected stable hash for the case-study receipt used in this paper is:

```
b83ee1178c4bea460b9c7e3c67d5ccf1d7ac330e017746236fcfc7274c459561
```

8.3 Tier 3: Full Case-Study Regeneration

To regenerate the MPRD case-study receipt from the archived evidence bundle:

```
python3 tools/run_neuro_symbolic_disaster_loop.py \
  --strict-research-gate \
  --json /tmp/neuro_symbolic_disaster_loop_latest.json \
  --md /tmp/neuro_symbolic_disaster_loop_latest.md
```

To materialize full per-hypothesis arrays:

```
python3 tools/run_neuro_symbolic_disaster_loop.py \
  --strict-research-gate \
  --full-results \
  --json /tmp/neuro_symbolic_disaster_loop_full.json \
  --md /tmp/neuro_symbolic_disaster_loop_full.md
```

The compact and full modes should agree on the stable semantic hash. If they do not, the compact receipt is not acceptable evidence.

No private hypothesis generator is required for these replication tiers. Private or experimental tools may help generate new what-if candidates, but public claims require deterministic receipts that can be checked from the archived artifact set.

8.4 Optional ZenoDEX Case-Study Replay

When the ZenoDEX repository or a Zenodo archive of it is available, the same paper claims can be inspected through its public replay surface:

```
git clone https://github.com/TheDarkLightX/ZenoDEX.git
cd ZenoDEX
python3 tools/permissionless_assurance.py status
python3 tools/build_stateful_disaster_search_expansion_plan.py --format json
python3 tools/sealed_bid_disaster_catalog.py
```

The expected public posture is not "all DEX behavior is proved." The expected posture is that the repository declares its promoted claim boundary, exposes replay commands for promoted claims, records disputed or experimental surfaces honestly, and makes named financial disaster witnesses executable or checkable under bounded assumptions.

9 What This Proves

The result proves bounded statements about a named witness language and evidence state.

9.1 Claim 1: Bounded No-Witness Result

Given the current case-study atlas, receipts, checker, generated hypothesis family, exhausted depth-11 frontier, and strict gate conditions, the latest run proves that no materialized generated hypothesis has a reachable disaster witness under the checker.

9.2 Claim 2: Fail-Closed Evidence Handling

The latest run proves that every implemented synthetic receipt, blocker, provenance, and optional-conflict mutation is handled according to the expected fail-closed policy. Missing mandatory evidence, failed receipts, artifact drift, unreferenced blocker symbols, stale hard provenance, and blocking optional receipts reject.

9.3 Claim 3: Research Permission Is Conditional

Research permission is not inferred from absence of a found bug. It requires absence of reachable witnesses, absence of unknown mandatory blockers, absence of optional research blockers, exhausted graph frontier, successful synthetic fail-closed checks, acceptable provenance, and a clean gate-critical checker.

9.4 Claim 4: Compact Receipt Soundness for This Gate

The compact and full receipt modes agree on a stable timestamp-free hash for the same semantic content. This supports compact archival without silently dropping gate-relevant content.

10 What This Does Not Prove

The method does not prove global software safety.

It does not prove:

- that the atlas contains every possible disaster state,
- that the generated witness language is complete,
- that all real executions are bounded by the modeled frontier,
- that every underlying fuzz or symbolic campaign was exhaustive,
- that every harness perfectly refines production behavior,
- that the checker has been machine-proved correct,
- that future code changes preserve the result,
- that cryptographic, network, OS, compiler, or deployment assumptions hold,
- that absence of a generated witness means absence of vulnerability.
- that the ZenoDEX case-study observations prove a live DeFi deployment safe without its own current release archive, configuration, keys, oracle assumptions, and replayed evidence.

It also does not give the LLM authority. The LLM or human proposer may suggest cases, but a case is only promotable after deterministic replay says it is promotable. Unknown is not a weak pass; it is a reject.

11 Design Principles

The technique suggests several reusable design principles.

11.1 Make Disaster States Concrete

Names such as `verify_fail_but_execute`, `duplicate_side_effect_after_claim`, and `unauthorized_policy_artifact_materializes` are better than generic "security bug" labels. Concrete names make harnesses, receipts, and blockers auditable.

11.2 Separate Witness Absence From Research Permission

No witness found is not the same as safe to research. Research permission also requires evidence freshness, optional-lane stability, clean checker state, and frontier exhaustion.

11.3 Treat Evidence as State

Receipts, provenance, optional guidance, and checker cleanliness are not bookkeeping outside the assurance protocol. They are state variables in the protocol.

11.4 Compile Verifier Structure, Not Trust

Verifier-compiler loops may produce quotients, summaries, controllers, or compact gates. These artifacts can reduce work, but final authority remains with the deterministic checker and replayable evidence.

11.5 Keep Private Search Separate From Public Evidence

Private tools may be useful for finding ideas, but public claims must be reconstructible from public artifacts, standard runtimes, and deterministic receipts.

12 Future Work

- Publish a generic schema for danger atlases, receipts, and what-if witness families independent of the MPRD codebase.
- Add graph-topology mutations: missing edge, inverted edge, duplicate edge, re-entry promoted to composition, and self-loop-at-surface.
- Upgrade all receipt schemas to carry provenance and content hashes.
- Connect selected gate claims to public Lean or SMT obligations where the checker logic is small enough to formalize.
- Generate minimal counterexample packets automatically when a reachable witness appears.
- Study obligation-targeted witness routing as a scheduling policy for LLM-assisted security review.
- Release a standalone ZenoDEX financial-risk case-study bundle with the same archival shape as the MPRD receipt bundle.

13 Code and Data Availability

The paper, renderer, and reference checker are part of the MPRD repository:

- Repository: https://github.com/TheDarkLightX/MPRD
- Reference checker: `tools/run_neuro_symbolic_disaster_loop.py`
- Paper source: `docs/whitepapers/NEURO_SYMBOLIC_DISASTER_LOOP.md`
- PDF renderer: `tools/render_neuro_symbolic_paper_pdf.py`
- Case-study receipt path: `docs/whitepapers/neuro_symbolic_case_study/neuro_symbolic_disaster_loop_latest.json`

The Zenodo archive DOI will be minted after the first tagged release associated with this repository. The repository README uses Zenodo's GitHub badge endpoint so the badge resolves to the latest DOI once available.

The ZenoDEX case study is described as a second application of the technique. Its full reproducibility depends on a separately released ZenoDEX repository or archive. The MPRD Zenodo archive should not be read as containing all ZenoDEX evidence unless those artifacts are explicitly included in the release payload.

14 Relationship to Formal Methods Philosophy Tutorials

This paper packages the following tutorial ideas into a software-hardening method:

- **Neuro-symbolic witness spaces:** LLMs and humans act as existential candidate generators; symbolic systems act as semantic filters.
- **Quantifier factoring:** universal assurance obligations become explicit counterexample searches over negated specifications.
- **Galois loops and obligation carving:** candidate spaces and obligation spaces are dual; progress should be measured on both sides.
- **Verifier-compiler loops:** repeated verifier behavior can sometimes be compressed into a quotient, repair coordinate, or controller, while final authority remains with the exact checker.
- **Loop-space geometry:** strong loops reshape the remaining search problem by changing witness language, stored state, ambiguity quotient, separator policy, and target artifact.
- **Counterexample-guided requirements discovery:** a counterexample may expose missing requirements, not merely a local defect; the loop must turn that into a new obligation rather than a false pass.

Reference URLs:

- Formal Methods Philosophy tutorials index
- Neuro-symbolic reasoning and witness spaces
- Quantifier factoring and neuro-symbolic loop engineering
- Galois loops and obligation carving
- Verifier-compiler loops
- Loop-space geometry
- Counterexample-guided requirements discovery
- Grammar-based fuzzing and structured search
- Concolic testing and branch exploration

15 Conclusion

What-if witness spaces change the shape of software hardening. They do not ask an LLM whether a system is safe. They ask creative systems to generate dangerous questions, then force every answer

through deterministic, receipt-backed, fail-closed checking.

The result is a bounded but honest assurance artifact. It can say what was searched, what was blocked, what evidence failed, what was compressed, which claim is open for research, and exactly where the claim stops. That boundary is the useful product. It is what lets high-stakes software learn, optimize, and promote new states without mistaking imagination for proof.